GRPO cheat sheet

Teach a Small AI to Think · unicodesnwrites.tech

GRPO (Group Relative Policy Optimization) comes from *DeepSeekMath* (Shao et al., 2024) and was used to train *DeepSeek-R1* (2025). Big idea: ask the same question several times, and grade each answer against its own group. No critic model needed.

One training step

#	Step	Tiny Adder	Real run (TRL)
1	Pick questions	16 random sums `a + b`	GSM8K word problems
2	Sample a **group** of G answers per question	`G = 8`	`num_generations=8`
3	**Grade** each answer with a rule	1 if `k == a + b`	format + correctness rewards
4	**Advantage** = (reward − group mean) / (group std + 1e-4)	`advantages(rs)`	`scale_rewards="group"`
5	**Nudge**: make positive-advantage answers more likely	`onehot(k) - softmax(L)`	backpropagation
6	Brakes: **clip** big moves, **KL** pull toward the start	`eps`, `beta`	`epsilon`, `epsilon_high`, `beta`

Advantages in a group of 8 (0/1 rewards)

Group	A for a right answer	A for a wrong answer
1 right, 7 wrong	+2.64	−0.38
4 right, 4 wrong	+1.00	−1.00
7 right, 1 wrong	+0.38	−2.64
all the same	0	0 (this group teaches nothing)

Advantages in a group always add up to zero.

The equation, in words

```
J(θ) = average over questions q and groups {o_1..o_G} from π_old of
  (1/G) Σ_i (1/|o_i|) Σ_t [ min( ratio·Â_i , clip(ratio, 1-ε, 1+ε)·Â_i ) - β·KL ]
ratio = π_θ(token) / π_old(token)      KL per token ≈ π_ref/π_θ - log(π_ref/π_θ) - 1
```

Symbol	Means
`π_θ`	the model being trained
`π_old`	the model that sampled this batch
`π_ref`	the frozen starting model
`Â_i`	group-normalized reward, shared by every token of answer i
`ε`	clip range (0.2 is common)
`β`	KL strength (TRL default 0.0)

Clipping, in one table (ε = 0.2)

Advantage	Ratio	Result
+	above 1.2	stop pushing (moved enough)
−	below 0.8	stop pushing (moved enough)
anything else		keep going (including corrections the other way)

Clipping only matters with more than one update per batch (`num_iterations > 1`).

Main TRL knobs (`GRPOConfig`, TRL 1.16)

Knob	What it does	Starting point on a free T4
`num_generations`	group size G	8
`per_device_train_batch_size` × `gradient_accumulation_steps`	completions per step (multiple of G)	8 × 2 = 16 (2 questions)
`max_completion_length`	thinking budget in tokens	256
`learning_rate`	step size	1e-5 with LoRA (TRL default 1e-6)
`beta`	KL leash	0 to 0.04
`epsilon` / `epsilon_high`	clip range / clip-higher	0.2 / 0.28
`loss_type`	how token losses are averaged	`"dapo"` (default), `"dr_grpo"`, `"grpo"`
`scale_rewards`	divide by group std?	`"group"`; `"none"` = Dr. GRPO style
`mask_truncated_completions`	ignore cut-off answers	`True`
`fp16` / `bf16`	precision	`fp16=True`, `bf16=False` on a T4

Reward functions: `def my_reward(completions, **kwargs) → list[float]`. Dataset columns arrive as keyword arguments (for example `answer`).

Later fixes worth knowing

Paper	Fix	TRL
DAPO (ByteDance Seed, 2025)	clip-higher	`epsilon_high`
	dynamic sampling (drop all-same groups)	not a single switch; watch `frac_reward_zero_std`
	token-level loss	`loss_type="dapo"`
	overlong shaping / filtering	`mask_truncated_completions`
Dr. GRPO (Liu et al., 2025)	remove length normalization	`loss_type="dr_grpo"`
	remove std normalization	`scale_rewards="none"`

Remember

- No disagreement in a group, no learning.
- Correctness should pay more than format.
- Always check held-out accuracy: the training reward can be hacked.